SAMPLE ASSIST · ENGINEERING · INTERN PROJECT

SA Marketplace — Delivery Report

Consumables ordering and marketplace for Collection Manager: the backend and the customer marketplace, seller portal and SA Admin front ends. What was built, why, how to run it, and what comes next.

Prepared for: Tech Lead · **Prepared by:** Rusira Senath Liyanage · **Status as at:** 25 October 2026 · **Source of truth:** Intern Project Brief, 15 July 2026

1. Summary

The brief asks for **one shared backend with several front-end entry points**: a marketplace for any buyer, a seller side with admin verification, and consumable ordering inside Collection Manager (CM). The Phase 1 backend for the marketplace, seller portal and admin app is complete, tested and documented, and the **customer marketplace, the seller portal and SA Admin front ends are built on top of it** and work end to end against the real API. The CM store front end has not been started. Nothing has been deployed; everything runs locally as the brief requires.

3	63	1,685	46
APIs on one database (customer, supplier, admin)	API operations (21 + 25 + 17)	automated tests (198 + 741 backend; 166 customer, 337 seller and 243 admin front end)	written decision records (ADRs) in the repo

Three things to know up front

1. **Two Phase 1 items are deliberately not built:** CM ordering (needs the CM front end and a decision on single sign-on) and adding stock to the buyer's inventory when an order is received. Both are recorded as open decisions, not forgotten (section 7).
2. **One deliberate departure from the brief's wording:** the catalogue requires a sign-in, so there is no anonymous browsing. Anyone can still register with an email and password, with no CM account needed (ADR 0021).
3. **Three of four front ends exist.** The marketplace for buyers, the seller portal and SA Admin are built and connected (section 3, "Front end"). The CM store is next (section 8). Prioritised recommendations and risks are in section 9.

What you are receiving

> **Latest source code:** the `phase-1-completed` branch of the Bitbucket repository `innomente/equipment-assist`. The backend lives in `SA_MarketPlace/Backend`, the customer front end in `SA_MarketPlace/Frontend/customer-facing-frontend`, the seller portal in `SA_MarketPlace/Frontend/seller-portal` and SA Admin in `SA_MarketPlace/Frontend/admin-portal`. Review and run that branch; section 9 recommends how to merge it into `main`.

Item	Where to find it
Source code	Branch `phase-1-completed`: `SA_MarketPlace/Backend` (three APIs and shared packages) `SA_MarketPlace/Frontend/customer-facing-frontend` (the buyer app) and `SA_MarketPlace/Frontend/seller-portal` (the supplier app)

Item	Where to find it
Setup instructions	Section 2 of this report, and the shorter `Backend/README.md` and the two front-end READMEs
Database: migrations, seed, checks	`packages/entities/src/migrations` (20), `packages/entities/src/seeds/seed.sql`, `scripts/verify-schema.sql`, `docker-compose.yml`
API documentation	Live Swagger at `/docs` on each app, and `Backend/docs/api-reference.md` with curl examples
Decisions and reasoning	`Backend/docs/adr` (46 records, listed in the appendix); front ends: `Frontend/customer-facing-frontend/docs` (`state-and-api.md`, `ui-build-plan.md`) `Frontend/seller-portal/docs` and `Frontend/admin-portal/docs` (each with `ui-build-plan.md` and `browser-checklist.md`)
Scope and data model	`CLAUDE.md` and `DATABASE.md` at the repository root
Tests	Unit tests beside the code (`*.spec.ts`); end-to-end tests in each app's `test` folder
This report	`Backend/docs/report` (PDF, HTML source and the script that renders it)

Not included: the screen designs for the four surfaces, which are kept in the project's separate design folders rather than the repository and should be shared with the team alongside the code; and any secrets. The only real credentials are the optional Gmail ones for sending email, which live in each developer's git-ignored `.env` (section 2.8). Every login is a fictional seeded account.

2. Set it up on your own machine

This is the whole path from an empty folder to a running system. Allow about 15 minutes, most of it the first `pnpm install` (it downloads Chromium for the invoice PDFs). Everything runs locally; nothing is installed system-wide except the three tools below.

2.1 What you need

Tool	Version	Notes
Node.js	18 or newer	Verified on 25.0.0
pnpm	10.11.0	Pinned in `package.json`. Run `corepack enable` and it installs itself
Docker Desktop	any recent	Must be **running** before any database command. No local Postgres is needed
Free ports	5433, 8001, 8002, 8003	Database, then customer, supplier and admin APIs

2.2 Get the code and install

1. **Clone the repository and open the backend folder**

```
git clone <repository-url>
cd <repository-folder>/SA_MarketPlace/Backend
```

2 **Create your environment file**

```
cp .env.example .env
```

The defaults work as they are for local use. `.env` is git-ignored and must never be committed. Replace `JWT_SECRET` if you share the instance.

3 **Install dependencies**

```
pnpm install
```

A note saying `@nestjs/core` build scripts were ignored is normal.

2.3 Database and Docker

One Postgres 16 container serves all three APIs. `docker-compose.yml` defines it, and its settings come from your `.env`.

Setting	Value	Why
Container	`sa-marketplace-db` (image `postgres:16-alpine`)	Small, current, and the same engine everywhere
Host port	**5433** (container 5432)	Avoids clashing with a Postgres you may already run on 5432
Credentials	`sa_user` / `sa_local_password`, database `sa_marketplace`	Local only; set in `.env`
Data volume	`sa_marketplace_pgdata`	Data survives `db:down`; `db:reset` destroys it on purpose
Health check	`pg_isready` every 5 s	"Running" is not "ready"; wait for *healthy*

4 **Start the database and wait until it is healthy**

```
pnpm db:up
docker compose ps        # STATUS should read "healthy"
```

5 **Create the tables by running the migrations**

```
pnpm --filter @sa-marketplace/entities migration:run
```

This runs the 20 migration files in `packages/entities/src/migrations` in timestamp order, one line per file ("… has been executed successfully"). The database is changed **only** this way, never by hand (ADR 0001). The result is 17 tables and 7 enum types.

6 **Load the demo data (the seed)**

```
pnpm db:seed             # prints: Seed applied successfully.
```

This runs `packages/entities/src/seeds/seed.sql` in one transaction, so a failure leaves the database untouched. It **replaces** the seeded tables, so running it again resets the demo data. Everything is fictional.

7 **Confirm the database is correct**

```
pnpm db:verify          # runs scripts/verify-schema.sql: every row must read PASS (33
checks)
pnpm --filter @sa-marketplace/entities db:check   # boots the database module and connects
```

The 33 checks cover the schema shape, the ban on storing card details, and the business rules the data must obey (for example, prices frozen on orders, unapproved sellers' products invisible).

What the seed loads	Count	What the seed loads	Count
Users	12	Products	40 (38 published, 1 draft, 1 retired)
Organisations	5	Categories (with sub-categories)	7
Supplier profiles	4 (approved, pending, rejected)	Orders, each with payment and delivery	7, one in every status
Carts / inventory rows	1 / 6	Invoices	0 (generated on first download)

2.4 Start the three APIs

Use one terminal for each. They run in watch mode and build the shared packages first.

```
pnpm customer-facing     # http://localhost:8001/docs
pnpm supplier-api        # http://localhost:8002/docs
pnpm admin-api           # http://localhost:8003/docs
```

2.5 Log in and try it (about five minutes)

Every seeded account uses the password `Password123!` (fake, local only).

App	Email	Use it to
Customer	`j.shaw@saclinics.com.au`	Buyer who is also a CM user: one account, two roles
Customer	`s.chen@miningco.com.au`	Buyer with order history
Supplier	`priya@bindimedsupply.com.au`	Approved seller (Bindi MedSupply)
Supplier	`r.patel@clinicalsupplies.com.au`	Seller still pending review (the "under review" case)
Admin	`a.morgan@sampleassist.com.au`	Approve or reject sellers, view all orders

1. On the customer Swagger page, call `POST /api/customer/v1/auth/login` with the buyer above, then paste the returned `access_token` into **Authorize**. Tokens last 15 minutes.
2. Browse with `GET /products` (try search, category, price range), add an item with `POST /cart/items`, and place the order with `POST /checkout`.
3. Pay with the mock card `4242 4242 4242 4242` (approved) or `4000 0000 0000 0002` (declined). Any future expiry and a 3-digit CVC work.
4. On the supplier page, log in as the Bindi seller, find the order under *orders requiring dispatch*, dispatch it, then download the invoice PDF from either app.
5. On the admin page, log in as the admin, open the supplier queue and approve or reject the pending seller.

2.6 Tests, and what to do when something goes wrong

```
pnpm test        # unit tests (198)
pnpm test:e2e    # end-to-end tests (741), against the real database
```

> **Run the end-to-end tests on a freshly seeded database.** They rely on the seed data, so manual testing that edits it (publishing the seeded draft product, changing a seeded password) makes a handful fail. `pnpm db:seed` resets it.

Symptom	Fix
Cannot connect to the Docker daemon	Start Docker Desktop and wait for it to settle
Missing required environment variable	Run `cp .env.example .env`
Port 5433 in use	Change `DB_PORT` in `.env`, then `pnpm db:reset`
Changed `DB_PASSWORD` and login fails	Docker reads it only on first start; run `pnpm db:reset`
401 after about 15 minutes	The token expired; log in again
429 on login	Limit is 5 attempts a minute per IP; wait a minute
Invoice PDF fails	Chromium missing: `npx puppeteer browsers install chrome`, then restart the API

Database commands at a glance: `pnpm db:up` start · `db:down` stop (data kept) · `db:reset` wipe and start clean · `db:seed` reload demo data · `db:verify` check · `db:logs` follow logs. A shell into the database: `docker compose exec postgres psql -U sa_user -d sa_marketplace`. For a GUI client use host `localhost`, port `5433`, user `sa_user`, database `sa_marketplace`. The same steps are kept short in `Backend/README.md`.

2.7 Run the customer front end

Needs Node 24 or newer (`nvm use` reads `.nvmrc`). With the customer API from 2.4 running:

```
cd ../Frontend/customer-facing-frontend
pnpm install
pnpm dev                  # http://localhost:5173
pnpm test                 # 166 tests; no backend needed
pnpm typecheck && pnpm lint && pnpm build
```

Sign in with the seeded buyer from 2.5, browse, add to the cart, and check out with `4242 4242 4242 4242` (approved) or `4000 0000 0000 0002` (declined). The API address is `VITE_API_URL` (`.env.example` has the default). The front end was installed, tested and built from a clean copy of its tracked files, and click-tested against a freshly seeded database.

The seller portal runs the same way, against the supplier API (`pnpm supplier-api` in `Backend/`):

```
cd ../Frontend/seller-portal
pnpm install
pnpm dev                  # http://localhost:5174
pnpm test                 # 337 tests; no backend needed
pnpm typecheck && pnpm lint && pnpm build
```

Seeded sellers (password `Password123!`): `priya@bindimedsupply.com.au` approved, `r.patel@clinicalsupplies.com.au` pending, `d.liu@cheaptestkits.com.au` rejected. `Frontend/seller-portal/docs/browser-checklist.md` is the walkthrough to run against a freshly seeded database.

SA Admin runs the same way, against the admin API (`pnpm admin-api` in `Backend/`):

```
cd ../Frontend/admin-portal
pnpm install
pnpm dev                  # http://localhost:5175
pnpm test                 # 243 tests; no backend needed
pnpm typecheck && pnpm lint && pnpm build
```

Sign in as the seeded admin `a.morgan@sampleassist.com.au` (password `Password123!`); a new admin account needs a `@sampleassist.com.au` address. The queue starts with pending suppliers to review. `Frontend/admin-portal/docs/browser-checklist.md` is the walkthrough to run against a freshly seeded database.

2.8 Real email (optional)

Reset-password emails are sent through a Gmail account when it is configured, and are only logged otherwise (ADR 0046). In Google Cloud create an OAuth client (Web application) with the redirect address `http://127.0.0.1:53682/callback`, enable the Gmail API, add the sending account as a test user, and put the client ID and secret in `Backend/.env` (`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`). Then run `pnpm mail:token`, sign in as the sending account, and restart the API; it saves the refresh token and sender address for you. While the OAuth client is in *Testing* mode the token expires after 7 days. Seeded addresses are fictional, so register a buyer with an inbox you own to see an email arrive.

3. What the brief asked and what was delivered

Section 3 of the brief defines Phase 1 as the committed deliverable. Each row below maps to it.

Brief item	Status	How it was delivered
Accounts and roles	Done	Registration and login on all three apps. One account can hold several roles (buyer, seller, collection manager, admin); the seeded buyer holds two. Each app accepts only its own tokens (ADRs 0017–0019).
Catalogue and products	Done	Sellers create, edit, publish and retire their own listings with images, category, GST-inclusive price and stock. Products are never hard-deleted because past orders refer to them (ADRs 0035, 0040).
Browse and search	Done	Keyword search, category and price-range filters, sorting, paging and a detail page. Only published listings from approved sellers appear.
Cart and checkout	Done	Cart with live prices and editable quantities. Checkout runs as one database transaction with a mock payment (ADRs 0023, 0024).
Orders	Done	A cart with several sellers becomes one order per seller. Buyers see their history and can reorder; sellers get an orders table with dispatch and cancel; admins see every order and can export CSV (ADRs 0022, 0036, 0042).
Electronic invoice	Done	A GST-inclusive tax invoice PDF with seller ABN, buyer details, lines, GST, totals, invoice number and date. It is generated once, stored, and re-downloadable by buyer, seller and admin.
Seller onboarding	Done	Sign-up with ABN (checksum-validated), business name and contact; pending, approved and rejected states each have their own response.

Brief item	Status	How it was delivered
Seller verification	Done	Admin queue with approve and reject (a rejection needs a reason the seller sees). Approval is the switch that makes listings visible (ADR 0041).
CM ordering	Not started	The backend already holds the shared account, orders and catalogue. It needs the CM front end and a decision on single sign-on.
Inventory update	Partly	Placing an order reduces the supplier's stock. Adding to the buyer's own inventory on receiving is deferred (section 7).
Shared data	Done (backend)	One database and one user table, so the same account, orders and inventory serve every entry point. It will be demonstrable end to end once CM is built.

Payments and invoices (brief section 4)

Checkout is a realistic mock: it collects the order, confirms it and marks it paid, with no gateway and no money moving. The mock follows how Stripe's test mode behaves (outcome decided by the card number). Card number, CVC and expiry are never stored, only the last four digits and the brand. Every order gets a tax invoice PDF as the brief requires.

Beyond the Phase 1 table

Built because the screens and the admin role needed them: password reset and change, seller dashboard (orders awaiting dispatch with an SLA countdown, listing counts), seller business profile, admin profile, and the admin orders CSV export. Seller ABN format validation (a Phase 2 item in the brief) is included because the sign-up form needs it.

Front end: the customer marketplace

Every customer screen in the designs is built and connected to the real API: catalogue with filters, search, sorting and paging; product detail with suggestions; cart; checkout (delivery and payment, with the declined-card pop-up); order confirmation; My orders with status filters and order-number search; order detail with invoice download and reorder; sign-in, registration, forgot and reset password, session-expired and sign-out; and My profile (details, change password).

Topic	What was done
Stack	React 19, Vite, TypeScript, Ant Design 6 and styled-components. State and API calls use Redux Toolkit with RTK Query, in the Collection Manager's feature-folder shape (`actions` , `slice` , `selector` , `types`). A store used by one page lives in that page's folder; a shared one in `src/store` .
API and sessions	One API client turns every failure into a single error shape and shows the backend's own messages. The access token lives in `sessionStorage` ; a rejected token ends the session and shows "Session expired"; signing out clears all cached data. Route guards send visitors to sign in and bring them back.
Where the backend won over the designs	GST is the total divided by 11; no supplier rating; password rule, card rules and order statuses as the backend defines them; no current-password field on change password (ADR 0030); no PO number; the invoice is offered only once an order has dispatched.
Backend changes it needed	An order-number `search` on `GET /orders` , the invoice file name exposed over CORS, and optional real email (ADR 0046). All are small, additive and tested.

Topic	What was done
Accessibility	Automated checks (axe) run on every page and pop-up; one h1 per page, page titles announced, a skip link, named dialogs, labelled fields, and text colours that pass the WCAG AA contrast minimum. The brand blue was darkened to `#0958d9` so white text on buttons passes (`CLAUDE.md` updated).
Pop-ups and unsaved changes	Every pop-up (clear cart, remove item, reorder, declined card, sign out, session expired) shares one layout: an icon, a one-line summary, what changes, what stays the same and equal buttons, with copy checked against what the backend does (for example, a declined card leaves the cart untouched). My profile asks "Save your changes?" before it is left with edits.
Quality	166 front-end unit tests (helpers, forms, components, pop-ups, the profile page); the pages and the Redux store have no automated tests and are checked in the browser against the real API, since the sample data and fake backend were removed. Lint, type check and build clean. A walkthrough against a freshly seeded database covered sign-in, shopping, cart, checkout (declined and approved), orders, invoice, reorder, profile (including a password change and restore), sign-out and registration.
Not offered	"Notify me" on out-of-stock products. It was built first, but with no backend it promised an alert that never came, so it was removed; out-of-stock products show a disabled button (section 7).

Front end: the seller portal

Every seller screen in the designs is built and connected to the supplier API: sign-in, registration, forgot and reset password; the three application screens (under review, not approved with resubmit, approved) and the gate that sends a seller to the right one; the dashboard with the orders waiting to go out and their SLA countdowns; orders (search, status filters, sorting, paging), the fulfilment screen with its pick list, dispatch, cancel and invoice download; the catalogue (search, filters, sorting, low-stock banner, retire; price and stock are read-only in the list and changed on the edit page); add and edit product with up to four images, Save draft and Publish; and the business profile (contact and fulfilment settings, change password, sign out).

Topic	What was done
Stack and structure	The same as the customer app (React 19, Vite, TypeScript, Ant Design 6, styled-components, Redux Toolkit with RTK Query, feature folders), with a left sidebar instead of a top bar and the seller's orange. About 35 generic files were copied rather than shared through a workspace package (decision recorded in its `ui-build-plan.md`); a shared package stays possible later.
Where the backend won over the designs	34 differences are listed in the plan's ledger, for example: sign-up has no "product categories" or "role" box (the profile derives categories from the seller's products), the status screens say "Sample Assist" not another company, dispatch sends no email, and sellers set their own prices directly.
Backend change it needed	One line: the supplier API exposes `Content-Disposition` over CORS so the invoice keeps its file name.
Colour and accessibility	White on the brand orange is only 2.4 to 1, so buttons use near-black text on orange and orange text is the darker `#ad4e00` . axe (WCAG 2.1 A and AA) found no violations on any page or pop-up after four fixes made in the final pass: unlabelled read-only fields, a details list without terms, links in running text told apart only by colour, and placeholder contrast. Below 992 px the sidebar becomes a menu button and drawer that a keyboard can reach.
Pop-ups and unsaved changes	Retire, cancel order, sign out and session expired share the same pop-up layout as the marketplace. The product form and the business profile ask "Save your changes?" (keep editing, discard, or save and go) when they are left with edits.

Topic	What was done
Quality	337 unit and component tests (the profile and product change detection, forms, tables, pick list, status gate, layouts); pages and the store are checked in the browser against the real API with the walkthrough in `docs/browser-checklist.md`. Lint, type check and build are clean.
Not built	Responsive page layouts (Phase 2); saving a half-filled product form across the 15-minute session; changing the sign-in email; restoring the pick list after a refresh (section 7).

Front end: SA Admin

Every admin screen in the designs is built and connected to the admin API: sign-in, creating an account (Sample Assist staff addresses only), forgot and reset password; the supplier verification queue with search, status tabs with counts, a banner and a sidebar badge; one application's review with the manual checklist and the approve and reject pop-ups; the all-orders list with payment and fulfilment filters, search, sorting, paging and Export CSV; an order's read-only detail with invoice download; and My profile (name, change password, sign out).

Topic	What was done
Stack and structure	The same as the seller portal, with its own purple and a sidebar of Suppliers (with a badge), Orders and My Profile. The shared files were copied again, so there are now three copies of about 35 files.
Where the backend won over the designs	20 differences are listed in the plan's ledger. The main ones: no "keep me signed in"; no invitation code on the register form (ADR 0038), where the backend instead requires a terms tick and a staff email domain; only the name is editable on the profile; no "Invite administrator" card; the order filters use the backend's real payment statuses and fulfilment stages; and the verification checklist is a screen-only aid.
Backend change it needed	One line: the admin API exposes `Content-Disposition` over CORS so the CSV export and the invoice keep their file names.
Decisions it makes	Approve and reject are the only actions: both email the applicant, neither can be undone here (there is no suspend), and the pop-ups say so. A decided application shows who decided it, when, and the reason the applicant was given. If another admin decides first, the screen shows the backend's 409 message and refreshes.
Accessibility	The purple `#722ed1` is 6.9 to 1 against white, so it needs no contrast workaround. axe (WCAG 2.1 A and AA) found no violations on any of 20 signed-in pages and pop-ups, nor on the sign-in, register, forgot, reset and session-expired pages. Skip link, one `h1` per page, brand-first titles, and a menu drawer below 992 px that a keyboard can reach.
Quality	243 unit and component tests (queue, filters and sort in the address, the decision pop-ups, order and profile pages, layouts); pages and the store are checked in the browser against the real API with the walkthrough in `docs/browser-checklist.md`. Lint, type check and build are clean. The walkthrough found two bugs the tests had missed (a previous record showing after a bad link, and a crash when the profile arrived after the page first rendered); both are fixed with regression tests, and the second was also fixed in the customer app.
Not built	Responsive page layouts (Phase 2); inviting administrators; suspending a supplier; a real check that an admin owns their mailbox (section 7).

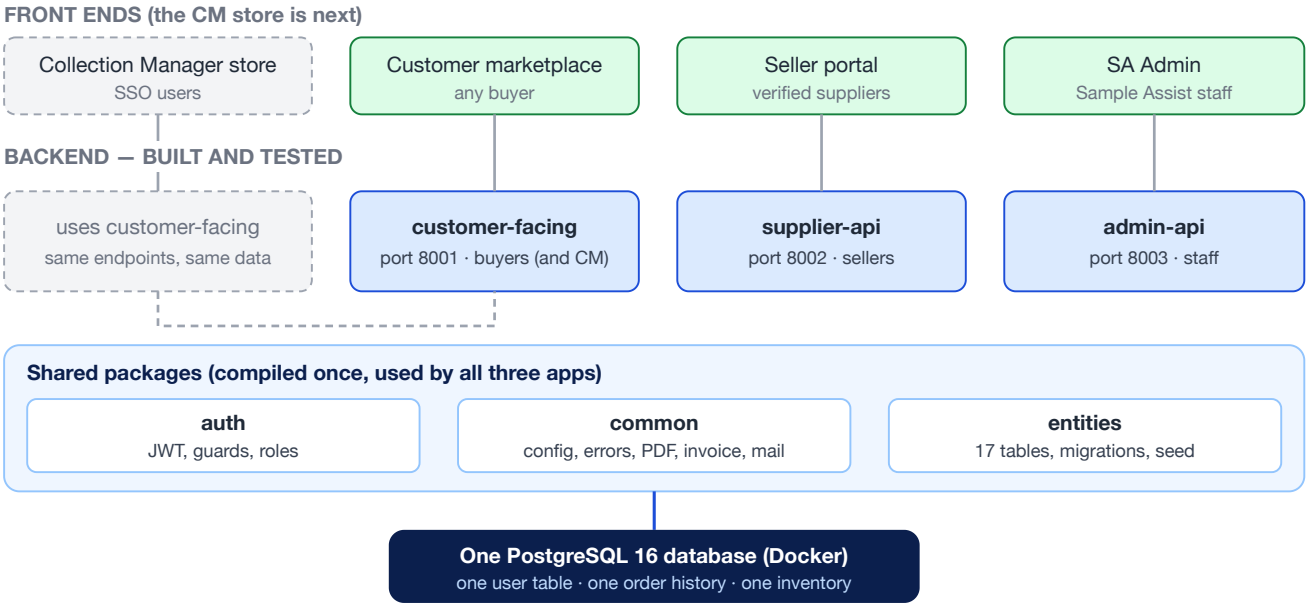
Working guardrails (brief section 7)

Guardrail	How it is met
Everything runs locally	Docker Compose for Postgres; no cloud. The one deliberate exception is the optional Gmail sending, off unless configured (ADR 0046)

Guardrail	How it is met
Never commit secrets	`.env` is git-ignored; only `.env.example` (local development values, blanks for the Google settings) is committed
No real payments or data	Mock checkout; all seed data is fictional
Validate input, hash passwords, guard every route by role, never expose one user's data to another	See section 6
Database changes via migrations	20 migrations; `synchronize` is off everywhere
Record decisions; keep the README current	46 decision records; a short README for a clean checkout, backend and front end
Runs from a clean checkout	Verified for the backend by installing, building, type-checking and linting a fresh clone, and migrating, seeding and testing a fresh scratch database; verified for the front end by installing, testing and building a clean copy of its tracked files

4. Architecture

The brief's most important rule is **one system with several entry points, not systems that sync**. The backend is a pnpm and Turborepo monorepo of three NestJS apps over shared packages and a single PostgreSQL database.



How the code is organised

- **One module per business area** inside each app (auth, products, cart, checkout, orders, catalogue, suppliers…), each with controller, service, DTOs and types, so a feature can be found and changed in one place.
- **Shared behaviour lives in packages**, not copied between apps: the one shared rule for "is this product orderable" is used by the catalogue, the cart and checkout.
- **The same order, invoice and stock rules serve every app.** The buyer and the seller download the very same stored PDF.

5. Decisions and why

Every decision has a short record in `docs/adr` (context, decision, cost). These are the ones that shape the system most. The **industry practice** column names the standard the choice follows.

Decision	Why	Industry practice followed	Record
One system, one database	The brief's core rule. A user who is both a CM user and a buyer must see one account, order history and inventory.	Single source of truth; avoids sync jobs and the data drift they cause.	Brief §2C
Schema changes only by migrations	Anyone can rebuild the exact database from nothing, and every change is reviewable.	Schema as versioned code (the Flyway and Liquibase approach); `synchronize` off.	0001
One account, many roles; per-app tokens	A person can be buyer and seller. A token for one app must not work on another.	Role-based access control; JWT `audience` claim; bcrypt (10 rounds) for passwords.	0017, 0019
Short tokens that can be revoked	Logging out or changing a password must stop old tokens at once, even though JWTs are stateless.	15-minute access tokens plus a server-side token version.	0018, 0030
The token decides whose data you see	Endpoints never take a user id. Services filter every query by the signed-in user, so no one can ask for another's data.	Prevents insecure direct object references (OWASP A01). Another user's record returns the same 404 as a missing one, so existence is not leaked.	0025
Money is a string, never a float	Floating point cannot hold cents exactly; GST and totals must reconcile to the cent.	Exact decimal arithmetic for money, with the database `numeric` type.	0008
Prices freeze at checkout; one order per seller	Editing a price must never rewrite past orders. Fulfilment, SLA and the tax invoice are all per seller (an invoice has exactly one seller).	Immutable order snapshots; Australian tax-invoice rules.	0022, 0023
Checkout is one transaction	Re-check the cart, reserve stock, then charge; a double click must not create two orders or oversell.	ACID transaction with row locks; order numbers from a database sequence, not `count()+1`.	0022, 0023
Invoices are stored documents	Seller and buyer details are copied onto the invoice when it is issued, so renaming a business never changes a past invoice. One PDF serves all three audiences.	Immutable financial records; own gapless-style sequence; an advisory lock so two simultaneous downloads cannot issue two invoices.	0036, 0043
Mock payment, no card data kept	Safe to run locally and meets the brief. Only last four digits and brand are stored.	PCI DSS (never store the CVC or full card number); modelled on Stripe's test cards. A database check fails if a card column ever appears.	0024
Password reset that leaks nothing	The same reply whether or not the email exists; links are single-use and short-lived.	256-bit random token, only its SHA-256 hash stored; anti-enumeration response; rate limiting on auth routes.	0029, 0020

Decision	Why	Industry practice followed	Record
Admin sign-up limited to staff domains	The brief has no admin invitation flow, and an open admin sign-up would be unsafe.	Allow-list by email domain. Honest limit: it is a speed bump, not proof of identity.	0038
Configuration from the environment	Local values stay out of git, and a missing variable fails at boot with a clear message.	Twelve-factor config; validate on start.	0002
Decisions written down	The brief asks for traceable decisions and a current README.	Architecture Decision Records (Nygard format).	docs/adr

Questions the brief left open, and how they were settled

Question	Answer and where it is recorded
Seller and buyer status words differ	One backend status, with the seller's five screens layered on top. A new order stays "new" until dispatched (ADR 0036).
Why a seller cancels an order	A fixed list of reasons (out of stock, cannot meet dispatch SLA, pricing error, other) (ADR 0036).
When a seller sees the "approved" screen	Once, on the first sign-in after approval, tracked on the profile (ADR 0032).
Admin invitations	Dropped; admin sign-up is domain-limited instead (ADR 0038).
How CM users sign in, and how CM inventory decreases	**Still open**; see section 7.

6. Quality and security

Testing

Level	Count	What it covers
Unit tests	191	Shared rules and helpers: auth service, invoice and PDF service (it kills a real Chromium to prove recovery), CSV, ABN and name validation, error filter
End-to-end tests	739	Every API route through HTTP against the real database: customer 117, supplier 360, admin 262. They cover auth, ownership, validation, checkout, invoices, and admin verification
Database checks	33	`db:verify` : schema shape, no card columns, and business rules in the data

Every suite passes on a freshly migrated and seeded database. I ran 736 of the 739 end-to-end tests against a scratch database so the working data stayed intact. The other three only assert that the database is named `sa_marketplace`, so they were run against the default database and also pass. Type checks and lint pass across all packages.

Security checklist

Concern	Control
Input validation	One global validation pipe: unknown fields are rejected, types are converted, text is trimmed; path parameters like order numbers are format-checked before any query

Concern	Control
Passwords and tokens	bcrypt hashing; signed JWTs with expiry and audience; revocation by token version; reset tokens stored only as hashes
Access control	Guards sit on whole controllers, so a new route cannot be added unprotected; role is checked as well as audience
Data isolation	Ownership is part of every query; suppliers see only their own listings and orders; competitors' data is never reachable
Abuse limits	Login, sign-up and reset are rate limited to 5 per minute per IP; JSON body size is capped; HTTP security headers via Helmet
Payments and privacy	No card data stored; fake seed data only; secrets in `.env`, never in git
Reliability	The PDF renderer restarts itself if Chromium dies, retries once, and times out instead of hanging

7. Known limitations and open items

Item	State and what is needed
CM ordering and single sign-on	Not built. Open question: does an SSO-provisioned user also get a password, or can the marketplace accept SSO for them? Needs the CM team's input and the CM front end.
Inventory on receiving	Not built. Nothing marks an order "received" yet, so the buyer's shelf stock never increases. Also open: CM stock can only go up unless CM already records consumption or we add a manual adjustment.
"Keep me signed in"	Not built by choice. Tokens last 15 minutes with no renewal; the front end shows "Session expired" and asks the buyer to sign in again. The design if revived is recorded in ADR 0045.
No anonymous browsing	The catalogue needs a sign-in (ADR 0021). Easy to relax for the public storefront if wanted.
Email is opt-in	Reset emails go through a Gmail account when configured (ADR 0046) and are logged otherwise. The Google OAuth client is in Testing mode, so its token expires every 7 days (run `pnpm mail:token` again). Gmail has sending limits; use a transactional provider for a real marketplace. A "your password was changed" email is not built.
Rate limits are in memory	Fine for one local instance; a shared store (Redis) would be needed to run several instances.
Local-development caveats	`.env.example` carries a development JWT secret (replace it for anything shared). Restarting dev servers can leave old Chromium processes running. The end-to-end tests expect seeded data.
Back-in-stock alerts	Not built. It needs a table of who is waiting, an endpoint, and an email when a supplier restocks (about half a day now that email exists). Until then out-of-stock products show a disabled button.
CM store front end	Not started. The customer marketplace, the seller portal and SA Admin have front ends.
Admin sign-up is a speed bump	Registration only checks the email's domain (ADR 0038); nothing proves the person owns the mailbox, so anyone can create `anything@sampleassist.com.au` and approve suppliers. Acceptable for local fake data, and it must be replaced before real use, for example by an existing admin approving each new admin or by a verified email.

Item	State and what is needed
Approving a supplier cannot be undone	There is no suspend or offboarding in Phase 1, so a wrong approval has no fix from the screens. The pop-up says so before it is confirmed.
SA Admin on small screens	The navigation adapts below 992 px but the pages are desktop-only, as in the seller portal. Responsive layouts are Phase 2.
Seller portal on small screens	The navigation adapts below 992 px but the pages are desktop-only: tables scroll sideways. A seller filling the long product form can also be signed out by the 15-minute session and lose it. Responsive layouts are Phase 2.
Pick list is not saved	The ticks live on the screen only; the backend has no status for "picking". A refresh forgets them.
Seeded product images	The seeded products point at image files that do not exist, so the browser logs a failed request for each and cards show a category icon instead. Products created through the seller API show real images.
Retiring a listing is final	In Phase 1 a retired product is read-only and cannot be published again.

8. Next steps

8.1 Front-end development (in progress)

The brief specifies React with Vite. The designs for all four surfaces already exist: 13 marketplace screens, 18 seller screens, 11 admin screens, and the CM store. Marketplace, seller and admin use Ant Design, each with its own accent colour; the CM store follows the existing CM look instead.

#	Step	State and what it involves
1	**Foundation**	Done Vite and React project, API client with token handling (Bearer header, a 401 ends the session), routing, layouts, shared components. The client is hand-written with typed endpoints rather than generated from Swagger.
2	**Customer marketplace**	Done All screens, connected to the API and tested (section 3).
3	**Seller portal**	Done All screens, connected to the supplier API and tested (section 3).
4	**SA Admin**	Done All screens, connected to the admin API and tested (section 3).
5	**CM store**	Not started Catalogue, cart, checkout and my orders inside Collection Manager, plus the SSO decision. Because the two buyer surfaces share no component library, each buyer feature is one backend ticket and two front-end tickets.

The seller portal and SA Admin copied the customer app's generic files instead of using a shared package, so about 35 files now exist three times. Decide whether to extract a shared package before the CM store makes it four, or to keep copying with identical file names and tests; a fix to a copied file goes into every copy (one bug fix already had to be made in two apps). Each feature keeps the brief's definition of done: a small, self-reviewed pull request; tests; clean lint and type checks; and it works end to end through the UI.

8.2 Finish the remaining Phase 1 items

- Decide and build **CM single sign-on** and the CM catalogue and ordering screens.
- Add **"mark as received"** so a delivered order increases the buyer's inventory, and settle how CM consumption is recorded.

- Add refresh tokens only if the 15-minute sign-out proves a problem in use.
- Build back-in-stock alerts if wanted, and a "your password was changed" email.

8.3 Later phases (from the brief, once Phase 1 is complete and solid)

Phase	Candidates
Phase 2 Marketplace depth	Richer filters and sorting (brand, test type, pack size, rating, newest, popularity); seller sales analytics and low-stock alerts; seller storefront pages, ratings and reviews; order tracking with status notifications; ABN lookup to help admin review; inventory reorder thresholds, purchase-order history and a stock movement log.
Phase 3 Nice to have	Redis caching for catalogue and search (the explicit bonus goal); real payment integration in test mode (for example Stripe test keys, never committed); moving email from Gmail to a transactional provider (only `MailService` 's body changes); discount codes and trade pricing tiers; an admin reporting dashboard.
Always out of scope	Real card processing, shipping-carrier integrations, mobile apps, multi-currency, and production deployment.

8.4 Handover

The brief asks for a final demo, handover notes and a short write-up of what is built, what is not, and known issues. This document is the write-up; the demo can follow the five-minute walkthrough in section 2.5.

9. Recommendations

In the order I would tackle them. The first four unblock the rest of the project.

#	Recommendation	Why, and who
1	**Merge `phase-1-completed` into `main` through a reviewed pull request**	The brief wants work to land on `main` by pull request and never broken. The branch now holds the backend and both built front ends; it is large, so review it in the commit order set out in `docs/report/pull-request-guide.md` rather than as one diff. Make the merge check: type check, lint, `db:verify` and every test suite on a freshly seeded database. *Tech lead.*
2	**Settle the two open decisions before building CM**	(a) How CM users sign in: does an SSO-provisioned user also get a password, or does the marketplace accept SSO? (b) Inventory: what marks an order "received", and how does CM record consumption, since stock can otherwise only go up. These shape the CM screens and the data flow. *Tech lead with the CM team.*
3	**Build the CM store next**	Three front ends now prove the structure and the API (section 8.1). Decide first whether they share a workspace package: three copies of about 35 files is where copying stops paying off, and a fourth would make it worse. None of the front ends has page tests (see Quality above), so a small end-to-end suite against the seeded API would close that gap for all of them. The CM store also needs recommendation 2 settled first.
4	**Use a separate test database**	The end-to-end tests expect the seeded data, so manual testing on the development database makes some fail, and one test asserts the database name. A dedicated test database (for example set through `DB_NAME`) keeps development data safe and makes the suites repeatable.

#	Recommendation	Why, and who
5	**Add automated checks when the team is ready**	The brief rules out CI/CD for now. Later, a pipeline running lint, type check, unit and end-to-end tests on a fresh database would have caught the one clean-checkout bug found during handover.
6	**Decide on anonymous browsing**	The catalogue needs a sign-in today (ADR 0021). If the public storefront should be browsable without an account, relax that rule and keep checkout signed in. *Product decision.*
7	**Tidy the development defaults**	Replace the development `JWT_SECRET` in `.env.example` with a placeholder, and have each developer generate their own.
8	**Revisit sessions now that the front end exists**	Add refresh tokens only if the 15-minute sign-out hurts. The design is recorded in ADR 0045.
9	**Handle the Google credentials safely**	Keep them only in `.env`. Delete any OAuth client whose secret was shared outside the team, publish the OAuth app or use a transactional provider if the 7-day token expiry becomes a nuisance, and never commit the refresh token.

Main risks

Risk	Level	Mitigation
CM sign-in and inventory rules are undecided	High	Resolve in recommendation 2 before CM screens are built; the backend already shares one account and one database.
Invoice PDFs depend on a local Chromium	Medium	The renderer restarts itself and retries (ADR 0043). Any future hosting must provide Chromium or move PDF rendering to a separate service.
Tests coupled to seeded data	Medium	Recommendation 4.
Rate limits are single-instance	Low	Fine locally; a shared store such as Redis is needed before running several instances.
Gmail sending depends on a Testing-mode OAuth token	Low	It expires every 7 days and Gmail has sending limits. Email is opt-in and failures never break a request; move to a transactional provider for real use (ADR 0046).
Admin sign-up only checks the email domain	Medium	Fine for local fake data; replace it before any real use (section 7, ADR 0038).
Three copies of the shared front-end files	Medium	Same names and tests everywhere, and a fix goes into every copy; decide on a shared package before the CM store (recommendation 3).
Knowledge held by one developer	Low	Reduced by 46 decision records, READMEs that run from a clean checkout, and this report.

Appendix: decision records

Each is a short file in `Backend/docs/adr` with context, the decision and its cost.

0001 Hand-written SQL migrations, not generated ones
0002 Local .env instead of AWS AppConfig
0003 Real HTTP status codes, not always-200
0004 Static module imports, not runtime discovery
0005 Intl.DateTimeFormat instead of moment
0006 A naming strategy instead of explicit column names
0007 uuidExtension: 'pgcrypto' for gen_random_uuid parity
0008 Money stays a string, never a float
0009 Timestamps declared on every entity, no base class
0010 The seed clears its tables and generates ids
0011 Lowercase email storage, not a functional index
0012 ON DELETE RESTRICT by default
0013 One validation pipe, no hand-written value conversion
0014 The exception filter keeps the status of exposed 4xx errors
0015 Middleware wildcard is '*' on Nest 10
0016 Each app owns its port; no shared PORT
0017 JWT access tokens via Passport, shared in packages/auth
0018 Logout bumps a token version on the user row
0019 Each app has its own token audience and allowed roles
0020 Login and sign-up are rate limited per IP, in memory
0021 The catalogue requires sign-in
0022 Order numbers come from a Postgres sequence, not count()+1
0023 Checkout re-validates the cart and reserves stock before charging
0024 Mock payment gateway: fixed test numbers, no card data stored
0025 Ownership checks return 404, not 403
0026 Reorder repopulates the cart; it never creates an order directly
0027 Profile enrichment stays app-side; packages/auth stays generic
0028 Profile edit excludes email, password and organisation
0029 Password reset: opaque hashed tokens, no enumeration, session-wide invalidation
0030 Change password: no current-password check, session survives its own token bump, real-length dots
0031 Seller registration: pending profile, split contact name, form fields without a home
0032 Seller application status: tracked dates, a persisted acknowledgement, resubmission
0033 Every date-time is a UTC instant; no bare date columns
0034 Seller dashboard: approval guard, order buckets, what the counts mean
0035 Seller catalogue: one product form, images in the same request, local image storage
0036 Seller orders: dispatch, cancel, and one shared invoice
0037 Seller business profile: what can be edited, the password rule, names that must fit
0038 Admin auth: sign-up limited to allowed email domains, no invitations
0039 Admin profile: the name only, the password rule, and a recorded last sign-in
0040 Product categories are worked out from a supplier's products
0041 Admin supplier verification: the queue, the application screen and the decision
0042 Admin orders oversight and the CSV export
0043 The PDF browser relaunches and retries when Chromium dies
0044 Swagger fields are declared explicitly where auto-detection fails
0045 "Keep me signed in" (refresh tokens) is deferred, with the design if revived
0046 Real email through Gmail: opt-in, never waited for, never logs the link